

Имплементација погона за дводимензионалне видео игре користећи *EKS* архитектуру и програмски језик *Rust*

Никола Огњеновић

Факултет техничких наука у Новом Саду

10.09.2026.

Првобитна идеја и погони за видео игре

- Погон за игре: софтверски систем који обезбеђује заједничку инфраструктуру за развој видео игара (рендеровање, ресурси, улаз, звук, физика...)
- Првобитна идеја: „Израда погона за видео игре користећи *Rust* и зашто је то лоша идеја”
- План: направити комплетан погон, укључујући и графички слој ниског нивоа, потпуно у *Rust*-у

ЕКС архитектура и зашто се користи

- Ентитети су само идентификатори, компоненте чисти подаци, а системи држе логику обраде
- Композиција уместо наслеђивања: нови тип објекта је нова комбинација компоненти
- Предности: мања спрегнутост, лакше проширење, боља искоришћеност кеша

ОО наслеђивање:
GameObject → *Character* →
Enemy → *FlyingEnemy* → *BossEnemy*

Дубока хијерархија

ЕКС композиција:
Transform + *Renderable* +
Health + *FlyingBossAI*

Равна флексибилна комбинација

Хипотеза

- Могуће је развити функционалан *ЕКС* погон за 2Д видео игре искључиво у безбедном *Rust* коду
- Циљ: *ЕКС* језгро (ентитети, компоненте, системи), рендеринг слој и управљање ресурсима
- Фокус рада: провера техничке изводљивости и анализа последица по архитектуру, једноставност и перформансе

Зашто безбедан *Rust*?

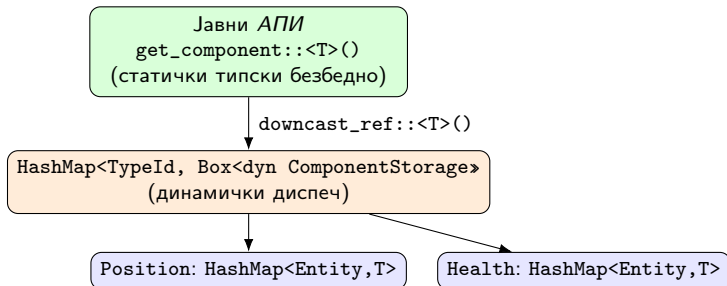
- Меморијска безбедност кроз власништво, позајмљивање и животне векове, грешке се хватају у компајлирању
- Избегавају се класични проблеми *C/C++* приступа (*dangling pointers*, *double free*, *data race*)

Ентитети: `id + generation`

- Ентитет је пар (`id`, `generation`)
- Идентификатори се рециклирају (`free_ids`)
- Генерација решава проблем висећих референци на обрисане ентитете
- Наспрам једног `u64`: боља читљивост и кеш понашање (независна обрада `id`-ева и генерација)
- Наспрам `UUID`: 2x мањи меморијски трошак и боља кеш локалност

Компоненте: складишта и *type erasure*

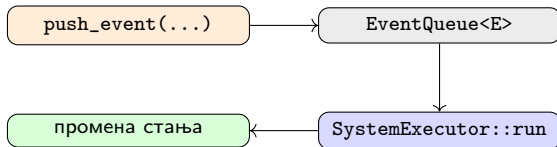
- Свака компонента (Position, Health, ...) има засебно, типизовано складиште
- Проблем: ComponentManager мора да чува складишта различитих типова у истој колекцији



Спољашњост остаје безбедна и статичка, а *type erasure* је сакривен унутар менаџера.

Догађаји и системи

- Догађај (нпр. `AttackEvent`) описује *намеру* акције, не мења стање сам по себи
- Промену стања врши тек систем који преузме догађај, у оквиру `SystemExecutor`-а
- Системи се извршавају детерминистички и секвенцијално



World: интеграциони слој и пример игре

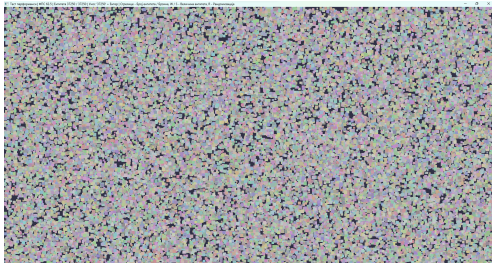
- World обједињује EntityManager, ComponentManager, EventManager
- Подржана серијализација/десеријализација стања у JSON формату
- Демонстрација: текстуална потезна борбена игра где су играч и непријатељи ентитети, без класних хијерархија

Рендеринг слој

- Изабран *wgpu*: мултиплатформски, небезбедан код остаје енкапсулиран унутар библиотеке
- Не намеће сопствени *game loop* нити фиксну организацију сцене и лако се уклапа у *EKC* систем
- Повезан са *EKC*-ом преко компонената `Transform2D` и `SpriteComponent`

Евалуациона апликација и резултати

- Стрес тест: велики број ентитета (квадрата) који се насумично крећу по прозору
- 37.000 ентитета у стабилних 60 *ФПС*
- 500.000 ентитета у ~ 2 *ФПС*, уз само $\sim 25\%$ *GPU* и $\sim 10\%$ *CPU* искоришћења $\rightarrow$ простор за паралелизацију



Закључак

- Хипотеза потврђена: могуће је направити безбедан *EKS* и рендеринг слој користећи *Rust* и *wgpu*
- Резултат: модуларан, читљив и меморијски безбедан погон за 2Д игре
- Даљи рад: паралелизација, софистицираније складиштење компоненти, скриптовање (нпр. *Lua*), физика (*Rapier*)
- Захвалница и питања